JavaScript is disabled on your browser.

compact1, compact2, compact3

java.util.logging

Class Level

java.lang.Object
    java.util.logging.Level

All Implemented Interfaces:

Serializable

```
public class Level
extends Object
implements Serializable
```

The Level class defines a set of standard logging levels that can be used to control logging output. The logging Level objects are ordered and are specified by ordered integers.
Enabling logging at a given level also enables logging at all higher levels.

Clients
should normally use the predefined Level constants such as Level.SEVERE.

The levels in descending order are:

- SEVERE (highest value)
- WARNING
- INFO
- CONFIG
- FINE
- FINER
- FINEST (lowest value)

In addition there is a level OFF that can be used to turn off logging, and a level ALL that can be used to enable logging of all messages.

It is possible for third parties to define additional logging levels by subclassing Level. In such cases subclasses should take care to chose unique integer level

values and to ensure that they maintain the Object uniqueness property across serialization by defining a suitable readResolve method.

Since:

1.4

See Also:

Serialized Form

Field Summary

Fields

Modifier and Type	Field	Description
static Level	ALL	ALL indicates that all messages should be logged.
static Level	CONFIG	CONFIG is a message level for static configuration messages.
static Level	FINE	FINE is a message level providing tracing information.
static Level	FINER	FINER indicates a fairly detailed tracing message.
static Level	FINEST	FINEST indicates a highly detailed tracing message.
static Level	INFO	INFO is a message level for informational messages.
static Level	OFF	OFF is a special level that can be used to turn off logging.
static Level	SEVERE	SEVERE is a message level indicating a serious failure.
static Level	WARNING	WARNING is a message level indicating a potential problem.

Constructor Summary

Constructors

Modifier	Constructor	Description

protected	**Level**(**String** name, int value)	Create a named Level with a given integer value.
protected	**Level**(**String** name, int value, **String** resourceBundleName)	Create a named Level with a given integer value and a given localization resource name.

Method Summary

Instance Methods

Concrete Methods

Modifier and Type	Method	Description
boolean	**equals**(**Object** ox)	Compare two objects for value equality.
String	**getLocalizedName**()	Return the localized string name of the Level, for the current default locale.
String	**getName**()	Return the non-localized string name of the Level.
String	**getResourceBundleName**()	Return the level's localization resource bundle name, or null if no localization bundle is defined.
int	**hashCode**()	Generate a hashcode.
int	**intValue**()	Get the integer value for this level.
static **Level**	**parse**(**String** name)	Parse a level name string into a Level.
String	**toString**()	Returns a string representation of this Level.

Methods inherited from class java.lang.Object

clone, finalize, getClass, notify, notifyAll, wait, wait, wait

Field Detail

OFF

```
public static final Level OFF
```

OFF is a special level that can be used to turn off logging. This level is initialized to `Integer.MAX_VALUE`.

SEVERE

```
public static final Level SEVERE
```

SEVERE is a message level indicating a serious failure.

In general SEVERE messages should describe events that are of considerable importance and which will prevent normal program execution. They should be reasonably intelligible to end users and to system administrators. This level is initialized to `1000`.

WARNING

```
public static final Level WARNING
```

WARNING is a message level indicating a potential problem.

In general WARNING messages should describe events that will be of interest to end users or system managers, or which indicate potential problems. This level is initialized to `900`.

INFO

```
public static final Level INFO
```

INFO is a message level for informational messages.

Typically INFO messages will be written to the console or its equivalent. So the INFO level should only be used for reasonably significant messages that

will make sense to end users and system administrators. This level is initialized to `800`.

CONFIG

```
public static final Level CONFIG
```

CONFIG is a message level for static configuration messages.

CONFIG messages are intended to provide a variety of static configuration information, to assist in debugging problems that may be associated with particular configurations. For example, CONFIG message might include the CPU type, the graphics depth, the GUI look-and-feel, etc. This level is initialized to `700`.

FINE

```
public static final Level FINE
```

FINE is a message level providing tracing information.

All of FINE, FINER, and FINEST are intended for relatively detailed tracing. The exact meaning of the three levels will vary between subsystems, but in general, FINEST should be used for the most voluminous detailed output, FINER for somewhat less detailed output, and FINE for the lowest volume (and most important) messages.

In general the FINE level should be used for information that will be broadly interesting to developers who do not have a specialized interest in the specific subsystem.

FINE messages might include things like minor (recoverable) failures. Issues indicating potential performance problems are also worth logging as FINE. This level is initialized to `500`.

FINER

```
public static final Level FINER
```

FINER indicates a fairly detailed tracing message. By default logging calls for entering, returning, or throwing an exception are traced at this level. This level is initialized to `400`.

FINEST

```
public static final Level FINEST
```

FINEST indicates a highly detailed tracing message. This level is initialized to `300`.

ALL

```
public static final Level ALL
```

ALL indicates that all messages should be logged. This level is initialized to `Integer.MIN_VALUE`.

Constructor Detail

Level

```
protected Level(String name,
                int value)
```

Create a named Level with a given integer value.

Note that this constructor is "protected" to allow subclassing. In general clients of logging should use one of the constant Level objects such as SEVERE or FINEST. However, if clients need to add new logging levels, they may subclass Level and define new constants.

Parameters:

`name - the name of the Level, for example "SEVERE".`

`value - an integer value for the level.`

Throws:

`NullPointerException - if the name is null`

Level

```
protected Level(String name,
                int value,
                String resourceBundleName)
```

Create a named Level with a given integer value and a given localization resource name.

Parameters:

name - the name of the Level, for example "SEVERE".

value - an integer value for the level.

resourceBundleName - name of a resource bundle to use in localizing the given name. If the resourceBundleName is null or an empty string, it is ignored.

Throws:

NullPointerException - if the name is null

Method Detail

getResourceBundleName

public String getResourceBundleName()

Return the level's localization resource bundle name, or null if no localization bundle is defined.

Returns:

localization resource bundle name

getName

public String getName()

Return the non-localized string name of the Level.

Returns:

non-localized name

getLocalizedName

public String getLocalizedName()

Return the localized string name of the Level, for the current default locale.

If no localization information is available, the non-localized name is returned.

Returns:

localized name

toString

```
public final String toString()
```

Returns a string representation of this Level.

Overrides:

`toString` in class `Object`

Returns:

the non-localized name of the Level, for example "INFO".

intValue

```
public final int intValue()
```

Get the integer value for this level. This integer value can be used for efficient ordering comparisons between Level objects.

Returns:

the integer value for this level.

parse

```
public static Level parse(String name)
                   throws IllegalArgumentException
```

Parse a level name string into a Level.

The argument string may consist of either a level name or an integer value.

For example:

- "SEVERE"
- "1000"

Parameters:

name - string to be parsed

Returns:

The parsed value. Passing an integer that corresponds to a known name (e.g., 700) will return the associated name (e.g., CONFIG). Passing an integer that does not (e.g., 1) will return a new level name initialized to that value.

Throws:

`NullPointerException` - if the name is null

IllegalArgumentException - if the value is not valid. Valid values are integers between Integer.MIN_VALUE and Integer.MAX_VALUE, and all known level names. Known names are the levels defined by this class (e.g., FINE, FINER, FINEST), or created by this class with appropriate package access, or new levels defined or created by subclasses.

equals

public boolean equals(Object ox)

Compare two objects for value equality.

Overrides:

equals in class Object

Parameters:

ox - the reference object with which to compare.

Returns:

true if and only if the two objects have the same level value.

See Also:

Object.hashCode(), HashMap

hashCode

public int hashCode()

Generate a hashcode.

Overrides:

hashCode in class Object

Returns:

a hashcode based on the level value

See Also:

```
Object.equals(java.lang.Object),
System.identityHashCode(java.lang.Object)
```


Submit a bug or feature
For further API reference and developer documentation, see Java SE Documentation. That documentation contains more detailed, developer-targeted descriptions, with conceptual overviews, definitions of terms, workarounds, and working code examples.
Copyright © 1993, 2026, Oracle and/or its affiliates. All rights reserved. Use is subject to license terms.
Also see the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.